GitHub : https://github.com/Hurakuto/tasks-api.git

Tous les screenshots sont dans le dossier « rendu ».

TP1

Combien de temps a duré le run ? Où voit-on les logs de chaque étape ?
Le run a duré 2s. Pour voir ce qui s'est passé à chaque étape, je vais dans l'onglet Actions, j'ouvre le run, puis je développe les étapes du job pour afficher leurs logs.

Quel test a échoué ? À quelle ligne ? Qu'est-ce qui a permis de le trouver rapidement ?
test_add_and_list_tasks et test_is_valid_title_accepts_normal_title sont les tests échoués, ligne 29 et 13.
La section short test summary info à la fin du rapport permet de les trouver rapidement.

Pourquoi attend-on que la CI soit verte avant de fusionner ?
Pour ne pas merge des erreurs.

Que se passe-t-il pour les étapes suivantes ?
Les étapes suivantes sont ignorées (*skipped*) : par défaut, GitHub Actions interrompt immédiatement le déroulement séquentiel du job. Les étapes *« Installer les dépendances »* et *« Lancer les tests »* ne sont donc pas exécutées (elles apparaissent grisées dans l'interface). Le job et le workflow passent au rouge (*Failed*) : le statut final de la CI est marqué en échec avec un code de sortie différent de 0. Seules les étapes conditionnelles s'exécutent.

TP 2

Quelle différence entre un linter (flake8) et un formateur (black) ? Pourquoi séparer lint et test en deux jobs ?
Un linter sert a détecter les erreurs alors qu'un formateur sert a vérifier l'uniformité du code. Pour détecter simplement l'erreur et éviter de chercher des problèmes précis avant d'avoir vérifié si le code est valide.

Pourquoi tester sur plusieurs versions de Python ? Que gagne-t-on avec le cache ? Un artefact, c'est quoi ?
Tester sur plusieurs versions de python permet d'éviter un bug sur une certaine version non vérifiée. Le cache permet de sauvegarder les dépendances déjà installées. Un artefact est un fichier qui se génère pendant l'exécution d'un workflow qui peut être téléchargé pour être relu plus tard.

Pourquoi interdire de pousser directement sur main ?
On interdit de push sur main pour éviter d'y ajouter des bugs grâce au ruleset.

Pourquoi Git ne peut-il pas résoudre ce conflit seul ? Qu'est-ce qui a vérifié le code après la résolution ?
Il ne sait pas ce qu'il doit être gardé.

Pourquoi GitHub masque-t-il le secret ? Car les secrets sont utilisés pour stocker des données sensibles telles que des mots de passe ou clé API Que se passe-t-il si on le commite dans le dépôt ? Il restera disponible dans les logs et sera retrouvable Secret ou variable : comment choisir ? Si l'information qu'on veut stocker est une information sensible, comme un mot de passe ou clé API, il faut choisir secret pour qu'elle soit cachée de toute personne regardant le repository. Si c'est juste une valeur qu'on veut réutiliser au sein du projet dont la propagation ne causerait pas de risque, on choisit variable.

Pour chaque pipeline : quel était le bug, comment l'avez-vous trouvé, comment l'éviter à l'avenir ?

A.
Problème d'indentation, le pipeline n'a pas pu démarrer, en relisant le fichier une fois terminé.

B.
La run démarre sans récupérer le code, le job échoue dès l'étape pip install, prendre le réflexe de mettre tout le temps actions/checkout en première étape.

C.
Les versions dans la matrice étaient déclarées sans guillemets, la run de la matrix affichait un job nommé test (3.1) au lieu de test (3.10), il faut toujours entourer les numéros de versions de guillemets.

TP 3

Différence entre une image et un conteneur ? À quoi sert le .dockerignore ?
Une image est une classe, un conteneur est une instance. Le .dockerignore sert a exclure des fichiers lors d'une contruction d'une image docker.

Que gagne-t-on à décrire le lancement dans un fichier plutôt que dans une longue commande ?
C'est plus simple, ça fait gagner du temps et on est sûrs de refaire la même chose a chaque fois.

Pourquoi build attend-il test ? Pourquoi éviter le tag latest en production ?
A cause de la ligne « needs : test ». On évite le tag latest car ça ne prouve pas la stabilité de la version.

Quel est l'avantage d'une image versionnée par rapport à « le code de ma machine » ?
Elle élimine le problème du « ça marche sur ma machine » en figeant le code, l'OS et les dépendances pour qu'ils tournent à l'identique partout.

À quoi sert un test de fumée ? Pourquoi demander une approbation humaine avant la production ?
À vérifier rapidement que l'application démarre bien, qu'elle écoute sur son port réseau et qu'elle ne crash pas immédiatement après le déploiement. Pour servir de garde-fou : cela évite les déploiements accidentels.

Pourquoi ne pas exécuter une application en root dans un conteneur ?
Un utilisateur standard limite les dégâts : il ne peut ni modifier les fichiers système, ni installer de paquets, ni compromettre les autres processus.

TP 4
J’ai fait le TP4 sur la branche TP3 par oublie de changer de branche.

C'est quoi un pod ? Un deployment ? Pourquoi un namespace ?
Un pod est la plus petite unité de Kubernetes contenant les conteneurs. Un deployment gère leur cycle de vie (nombre de répliques et mises à jour sans coupure). Un namespace isole les ressources dans le cluster (comme un dossier).

Pourquoi sépare-t-on le dépôt de l'application et le dépôt de configuration ?
Pour éviter les boucles infinies de CI (modifier un manifeste ne relance pas les tests du code) et séparer les droits d'accès entre développeurs et ops.

Que signifie selfHeal ? Pourquoi le retour arrière est-il simple en GitOps ?
selfHeal annule toute modification manuelle pour remettre l'état décrit dans Git. Le retour arrière est simple car un simple git revert sur Git force Argo CD à réappliquer immédiatement l'ancienne version.

Résumez en 5 lignes le chemin d'un git tag jusqu'au pod. Qui pousse (push) et qui tire (pull) ? Quelle différence avec un déploiement CI/CD « classique » ?

On pousse un tag v0.2.0 sur le dépôt de code tasks-api.

- GitHub Actions build et publie la nouvelle image Docker sur GHCR.
- Le pipeline modifie la version dans le dépôt tasks-config et le pousse.
- Argo CD détecte le changement sur le dépôt tasks-config.
- Argo CD applique la mise à jour dans Kubernetes, qui lance les nouveaux pods.